



UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
BACHARELADO EM TECNOLOGIA DA INFORMAÇÃO
VISÃO COMPUTACIONAL

RELATÓRIO DA TERCEIRA UNIDADE

DAVI DIÓGENES FERREIRA DE ALMEIDA

NATAL/RN
06/12/2025

1. Introdução

O tênis destaca-se como um dos esportes que mais utilizam estatísticas avançadas, com organizações como ATP, WTA e os jogadores se aproveitando desses dados para melhoria da performance em quadra. Essa geração massiva de dados tabulares e visuais, superior a outros esportes em granularidade pública, permite análises profundas sobre estratégias, movimentação e qualidade de golpes. O projeto crowdsourced Match Charting Project (MCP) coletou dados shot-by-shot de milhares de partidas profissionais, totalizando mais de 5.000 partidas detalhadas com direções, profundidades e tipos de erros.

Nesse contexto, o aprendizado por reforço surge como uma abordagem promissora para explorar esses dados e otimizar estratégias, treinando agentes em ambientes simulados que replicam decisões táticas como escolha de golpes e direções, visando maximizar a probabilidade de vencer pontos, games e partidas. Ao aprender políticas a partir de recompensas relacionadas a vitórias e eficiência de golpes, esses agentes podem sugerir ajustes táticos finos que complementam a análise estatística tradicional e apoiam o processo de tomada de decisão de jogadores e treinadores.

2. Fundamentação Teórica

2.1. Conjunto de dados

O Match Charting Project, de Jeff Sackmann, fornece dados shot-by-shot de milhares de partidas profissionais, incluindo direções, tipos de erros e profundidades de retornos. Esses arquivos CSV, separados por gênero e com metadados de torneios, serviram de base para a construção do ambiente em que foram experimentados algoritmos de aprendizado por reforço. No conjunto de dados, cada ponto de uma partida é codificado com a sequência de golpes efetuados, por exemplo: `4f18b1f1b1b3f3f1s1f+3f1z1*`. Este formato de codificação armazena 18 variações de golpes e 3 variações de direção, além de códigos para erros, winners e outros movimentos específicos.

2.2. Construção do ambiente

O objetivo desta etapa é construir um ambiente de uma partida de tênis em um formato similar à API da biblioteca Gymnasium, de modo a permitir uma fácil integração com algoritmos de aprendizado por reforço já desenvolvidos ao longo do semestre. O ambiente deverá conter um estado com os seguintes atributos:

- Último golpe executado
- Direção do último golpe
- Pontuação no game dos dois jogadores
- Pontuação no set dos dois jogadores
- Quem está sacando no momento.

E as ações a serem executadas sobre o ambiente devem possuir o seguinte formato:

- Golpe a ser executado
- Direção do golpe a ser executado

2.2.1. Parsing dos pontos

A primeira etapa para a construção do ambiente é fazer o parsing dos códigos de golpes de cada ponto. Com isso, pretendemos separar um ponto em golpes, erros e winners, com o objetivo de, posteriormente, realizar a contagem de transições. O dataset é composto por tabelas em um formato similar a este:

Pt	Set1	Set2	Gm1	Gm2	Gm#	Svr	1st	2nd	PtWinner
38	3	5	15	40	9	1	4n	4b1b1b 3n@	1

Cada golpe usa 2 caracteres: letra + número. Exemplo: f1 (forehand para forehand de destro), b3 (backhand para backhand de destro) ou v2 (forehand volley no meio).

- f = forehand groundstroke; b = backhand groundstroke
- r = forehand slice; s = backhand slice
- v = forehand volley; z = backhand volley
- o = overhead/smash; p = overhead backhand
- u = forehand drop; y = backhand drop
- l = forehand lob; m = backhand lob
- h = forehand half-volley; i = backhand half-volley
- j = forehand swinging volley; k = backhand swinging volley
- t = trick shots; q = shot desconhecido
- 1 = para o lado direito do adversário
- 2 = no meio da quadra
- 3 = para o lado esquerdo do adversário

O objeto final desta etapa é uma tabela que lista todas as transições entre golpes e direções. Nesta fase também são filtrados caracteres de golpes e particularidades que não serão contabilizadas, como o “+”, que indica combos de saque e voleio e o “^”, que indica *drop volleys*.

game_n umber	shot_nu mber	server	player	last_shot_ type	last_shot_ dir	shot_type	shot_dir
1	1	1	1	#	1	serve	4
1	2	1	2	serve	4	b	1
1	3	1	1	b	1	b	1
1	4	1	2	b	1	b	3
1	5	1	2	b	3	@	3

2.2.2. Contagem de transições

Utilizando a tabela montada na etapa anterior, são contadas todas as transições de golpes, resultando na seguinte estrutura:

Golpe Anterior	Direção Anterior	Golpe Atual	Direção Atual	Contagem
#	1	serve	1	15497
f	2	b	1	6903

2.2.3. Construção do grafo de transição

O objetivo desta etapa foi transformar o conjunto de dados de contagem de transições de jogadas em um modelo estocástico que pudesse ser consultado em tempo real pelo ambiente de simulação. A estrutura do grafo de transição é fundamentalmente uma Cadeia de Markov de primeira ordem. Em uma Cadeia de Markov, a probabilidade de transição para o próximo estado depende unicamente do estado atual, e não da sequência de estados que o precederam. No nosso contexto, o estado é definido pela última jogada executada, uma tupla (tipo_de_jogada, direção). Por exemplo, ('f', 1) representa um forehand na direção 1.

Para a construção do grafo, foi criado um dicionário no formato `{(estado_origem, estado_destino): contagem}` com contagem inicializada em 0 para todas as possibilidades de transição. Aplicam-se regras lógicas do tênis para invalidar transições impossíveis, como:

- Um saque não pode ser precedido por outro saque.
- Uma jogada de erro ou winner deve ser seguida por um saque para iniciar o próximo ponto.
- Um saque só pode ocorrer após um erro ou winner.

Após essa inicialização, o dicionário foi preenchido com as contagens reais do arquivo de dados. Transições que não ocorreram nos dados mantiveram sua contagem em zero. O próximo passo foi converter essas contagens em uma distribuição de probabilidade. Este cálculo foi projetado para ser flexível através do uso de um parâmetro de temperatura (T). O cálculo é realizado da seguinte forma:

- Coleta de Contagens: Para um estado de origem, coletamos as contagens de todas as transições para os estados de destino possíveis.
- Aplicação da Temperatura: Cada contagem (c) é ajustada pela temperatura, resultando em um "logit" ajustado: $\text{logit_ajustado} = c \wedge (1 / T)$
- Normalização: Os logits ajustados são então normalizados para que a soma de todas as probabilidades de transição a partir do estado de origem seja igual a 1. Isso é feito dividindo cada logit ajustado pela soma de todos os logits ajustados.

O parâmetro de temperatura controla o nível de aleatoriedade do oponente. Para a nossa simulação inicial, utilizamos $T = 1.0$ para refletir fielmente a distribuição dos dados:

- $T = 1.0$: As probabilidades são diretamente proporcionais às contagens observadas.
- $T > 1.0$: A distribuição de probabilidade se torna mais uniforme, aumentando a aleatoriedade e a chance de o oponente escolher jogadas menos frequentes.
- $T < 1.0$: A distribuição se torna mais "aguda", fazendo com que o oponente escolha as jogadas mais frequentes de forma quase determinística.

2.2.4. Definição de regras do tênis

Após montado o modelo de transições de golpes do ambiente, é necessário aplicar este modelo às regras do esporte para que as ações sejam afetadas pelas particularidades de uma partida completa. Esta etapa implica na construção de uma classe que será responsável pela contagem de pontos e games, cuidando de lógicas como a pontuação “15”, “30” e “40” e a administração de “vantagem”. A interação com esta classe será feita após a finalização de um ponto por meio da função `point(player)`, que recebe o identificador do jogador que pontuou e retorna uma tupla com algumas informações sobre o estado da partida: (`pontuação no game`, `pontuação na partida`, `vencedor do game`, `vencedor do set`). Estas informações são utilizadas posteriormente para a montagem do estado do ambiente no momento do ponto.

Além disso, a classe do ambiente é responsável por identificar ações ilegais no contexto de uma partida de tênis, são essas: Saque durante um ponto e golpe normal no início de um ponto. Ações inválidas são tratadas de modo que o ponto só avança se for executada uma ação legal, o que gerou alguns loops infinitos em rodadas de avaliação.

2.2.5 Definição de recompensas

As recompensas foram definidas sobre 8 fenômenos de uma partida e ajustadas ao longo da experimentação:

- Ação executada = -0.1
- Ação inválida = -0.5
- Ponto ganho = 1
- Ponto perdido = -1
- Game ganho = 2
- Game perdido = -2
- Partida ganha = 10
- Partida perdida = -10

2.2.6. Ambiente Resultante

Como resultado, obtivemos um ambiente em moldes similares ao da biblioteca Gymnasium, de modo a facilitar a implementação de algoritmos e permitir o reaproveitamento de código produzido ao longo da disciplina.

```
> env.state
State(last_shot_type='winner', last_shot_direction=2,
player_game_score='15', player_set_score=3, pc_game_score='0',
pc_set_score=1, player_serves=True)
```

O estado, no trecho acima, está representado em sua forma legível. Na hora de enviar para o modelo, os *shot_types* e as *shot_directions* são codificados com *one hot encoding* e os *game_scores* transformados de ‘0’, ‘15’, ‘30’, ‘40’, ‘AD’ para 0,1,2,3,4. Este encoding resulta em um vetor de 29 inteiros para representar o estado completo.

```
> print(env.step("b", 2))
(State(last_shot_type='#', last_shot_direction=2, player_game_score='0',
player_set_score=3, pc_game_score='15', pc_set_score=1,
player_serves=True), -5, False, {})
```

2.3. Algoritmos

Para os experimentos, foram utilizados 2 algoritmos de aprendizado por reforço, sendo estes o Deep Q-Learning e o Reinforce. **Falar sobre por que eles foram escolhidos.**

2.3.1 Deep Q-Learning

O Deep Q-Learning é um algoritmo baseado em valor que combina o Q-Learning tradicional com redes neurais profundas para aproximar a função de valor $Q(s,a)$. Em espaços contínuos ou de alta dimensão, tabelas Q tornam-se inviáveis devido à explosão combinatória. Deep Q-Networks (DQN) substituem tabelas por redes neurais profundas que mapeiam estados para Q-valores de todas as ações, podendo usar convoluções e outras técnicas de deep learning para processar entradas como pixels e sequências.

Este método utiliza uma rede neural principal $Q(s,a;\theta)$, que estima Q-valores para todas ações a partir do estado s , e uma rede alvo $Q(s,a;\theta^-)$, que gera alvos estáveis e é atualizada periodicamente para evitar divergência no treinamento. O processo de treinamento se inicia com a seleção e execução de ações no ambiente com a rede principal. Transições (s, a, r, s') são armazenadas em um buffer de replay.

A atualização de pesos é feita após o sorteio de mini-lotes aleatórios, quebrando correlações sequenciais e melhorando a generalização e estabilidade durante o gradiente descendente. O cálculo da função de perda é realizado de modo que:

$$alvo = r + \gamma \max_{a'} Q(s', a'; \theta^-)$$

$$L(\theta) = (alvo - Q(s, a; \theta))^2$$

Nesta operação, o valor máximo do estado seguinte (s') na rede alvo (θ^-) é multiplicado por um fator γ e somado à recompensa (r) para montar o valor alvo. Este valor alvo é subtraído do valor atual encontrado pela rede principal (θ), baseado no estado atual (s) e na ação atual (a), e, em seguida, elevado ao quadrado para encontrar o valor da perda.

Após o treinamento, a seleção de ações é feita enviando o estado para a rede neural e selecionando a ação de maior valor retornado.

2.3.2 Reinforce

O algoritmo REINFORCE é um método baseado em política, ou seja, permite treinar um agente através da atualização direta dos parâmetros da política, sem a necessidade de uma função de valor. Os parâmetros da política são atualizados na direção do gradiente calculado, de modo que aumentam a probabilidade das ações que levaram a retornos altos e diminuem a das que não foram proveitosas com o objetivo de maximizar sua recompensa ao longo de episódios.

O treinamento baseia-se na geração de episódios completos antes da atualização de pesos da política:

$$\tau = (s_0, a_0, r_1, \dots, a_{T-1}, r_T)$$

Após a geração de um episódio, são calculados os logaritmos das probabilidades das ações escolhidas. Além disso, um vetor G de recompensas é montado com a recompensa acumulada dos passos posteriores e descontada por um fator de declínio:

$$G_t = \sum_{k=t}^T \gamma^{k-t} r_k$$

Além disso, o vetor G é normalizado seguindo a fórmula $(G_t - G_t.mean()) / (G_t.std() + 1e-8)$ para tornar o treinamento mais estável e evitar gradientes explosivos. Para cada par $(\log_prob, G_t)$, computa-se a perda $-\log_prob * G_t$ para cada de passo tempo t , onde $\log$ -probabilidade negativa escalada pelo retorno reforça ações boas (alto R) e penaliza ruins (baixo R).

$$loss = \sum_t \log \pi(a_t | s_t) G_t$$

Após o treinamento, a seleção de ações é feita enviando o estado para a rede neural e sorteando, baseado nas probabilidades retornadas pela função softmax, uma ação a ser executada.

3. Metodologia Experimental

3.1. Agente DQN

O agente DQN foi implementado através da classe `DQNAgent`, utilizando duas redes neurais profundas idênticas, sendo uma rede principal, atualizada a cada batch sorteado, e uma rede alvo, atualizada em uma frequência definida em código:

Arquitetura da rede: Rede feed-forward com 3 camadas totalmente conectadas

- Camada de entrada: tamanho do estado codificado (29 dimensões)
- Camadas ocultas: 512 neurônios cada, com ativação ReLU
- Camada de saída: número de ações possíveis (54 ações = 18 tipos de jogada × 3 direções)

Hiperparâmetros iniciais:

- Taxa de aprendizado (α): 0.001
- Fator de desconto (γ): 0.95
- Epsilon inicial: 1.0
- Epsilon mínimo: 0.01
- Taxa de decaimento de epsilon: 0.999
- Tamanho do buffer de replay: 10.000 transições
- Tamanho do mini-lote: 32 experiências
- Frequência de atualização da rede alvo: a cada 100 steps
- Estratégia de exploração: Epsilon-greedy com decaimento exponencial, permitindo alta exploração no início do treinamento e convergência para exploração conforme o agente aprende.

3.2. Agente Reinforce

O agente REINFORCE foi implementado através da classe ReinforceAgent, utilizando uma rede neural de política:

Arquitetura da rede: Rede feed-forward com 3 camadas totalmente conectadas

- Camada de entrada: 29 dimensões (estado codificado)
- Camadas ocultas: 128 neurônios cada, com ativação ReLU
- Camada de saída: 54 neurônios com softmax, representando a distribuição de probabilidade sobre as ações

Hiperparâmetros principais:

- Taxa de aprendizado (α): 0.001
- Fator de desconto (γ): 0.99
- Estratégia de atualização: A política é atualizada apenas ao final de cada episódio, utilizando os retornos descontados normalizados para reduzir a variância do gradiente.

3.3. Ferramentas e bibliotecas

Todos os agentes e algoritmos foram implementados utilizando a linguagem Python com auxílio de pacotes.

3.4. Avaliação do Desempenho

As métricas avaliadas foram relativas tanto a operações gerais do aprendizado por reforço quanto a fatores específicos do ambiente trabalho, sendo estas:

- Perda média -> Treino
- Recompensa por episódio -> Treino e Avaliação
- Recompensa média dos últimos 100 episódios -> Treino
- Tamanho do episódio -> Treino e Avaliação
- Número de golpes inválidos do agente -> Treino e Avaliação
- Taxa de vitórias -> Treino e Avaliação

Para o caso específico do Deep Q-Learning, também foram observados os Q-Valores retornados pela rede neural.

Inicialmente, o principal valor observado e levado em consideração foi o número de golpes inválidos, considerando o objetivo de alcançar um agente que faz escolhas válidas e úteis para uma partida de tênis. Em casos que o modelo alcançou um número muito baixo ou zero de golpes inválidos, foram avaliadas outras métricas, como a recompensa, a perda e a taxa de vitórias alcançadas.

3.5. Métodos de visualização

O MLFlow disponibiliza formas de visualização facilitadas para todas as métricas monitoradas. Esta ferramenta foi a principal forma de observar e avaliar resultados de treinamentos e testes. Além disso, o ambiente foi configurado para retornar ações executadas e resultados no terminal de execução do código, o que auxiliou na percepção

da qualidade de escolha de ações e no *debug* de loops de ações inválidas e outros problemas que surgiram durante o desenvolvimento.

```
=== Playing match 1 ===
Vez do Turn.PC
Chosen next action: (winner, 1)
Chosen next action: (serve, 1)
PC fez um winner
Ponto para o PC
Turno para: Turn.PC
Chosen next action: (serve, 2)
Chosen next action: (f, 2)
Ponto continua, turno de: Turn.PLAYER
Shot 1: {'shot_type': 'o', 'shot_direction': 1, 'reward': -1.02, 'info': {}}
Match Score: Player 0-0 | PC 15-0
Vez do Turn.PLAYER
Chosen next action: (winner, 1)
Chosen next action: (serve, 3)
Player fez um winner
```

4. Resultados e Discussão

Os resultados sugerem que o DQN não conseguiu estabilizar a função de valor em um ambiente com espaço de ações grande, estrutura de recompensa densa e presença de ações ilegais que geram loops e trajetórias pouco informativas no replay buffer. A combinação de altas penalizações para episódios longos com a alta exploração faz com que o agente acumule retornos extremamente negativos, gerando alvos de grande magnitude e tornando o treinamento sensível à taxa de aprendizado. Um possível experimento futuro é uma normalização das saídas de valores similar ao que é feito com o REINFORCE, de modo a evitar a explosão de gradientes.

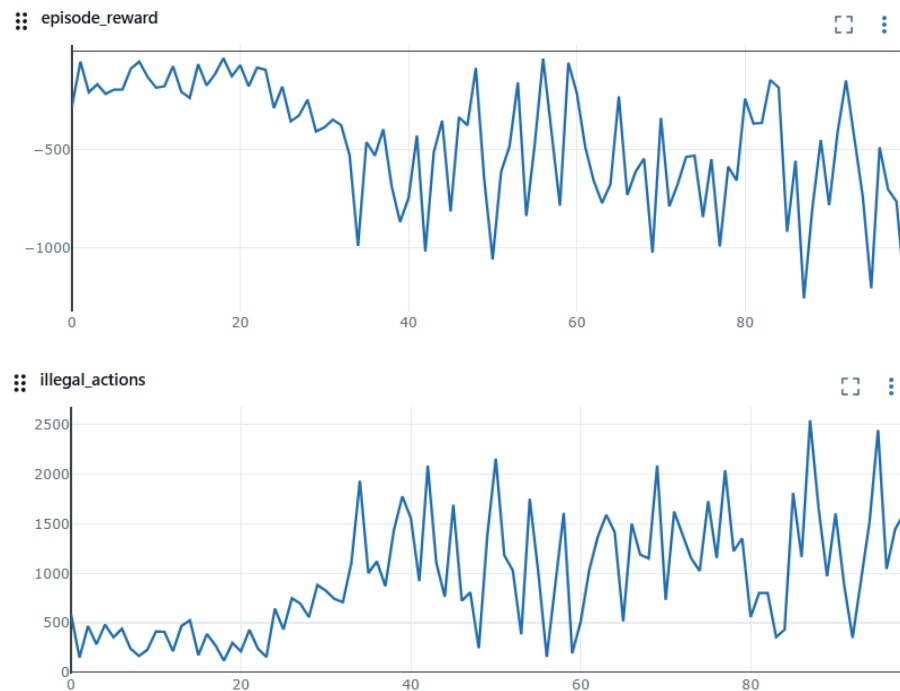
Enquanto isso, o REINFORCE conseguiu ajustar-se bem a partir do retorno de episódios completos, reduzindo naturalmente a probabilidade de ações ilegais e se adaptando ao grande espaço de ações de maneira bem consistente ao longo do treinamento. O agente REINFORCE, no geral, apresentou ótimos resultados, alcançando rapidamente políticas com alta recompensa média por episódio, baixa frequência de ações ilegais e taxa de vitória significativamente superior às obtidas pelo DQN. Esse resultado é coerente com a natureza do algoritmo, que lida bem com a estocasticidade do grafo de transições de golpes, permitindo aprender distribuições de ações que evitam golpes inválidos e exploram sequências táticas eficazes mesmo em um espaço de ações grande.

4.1. Deep Q-Learning

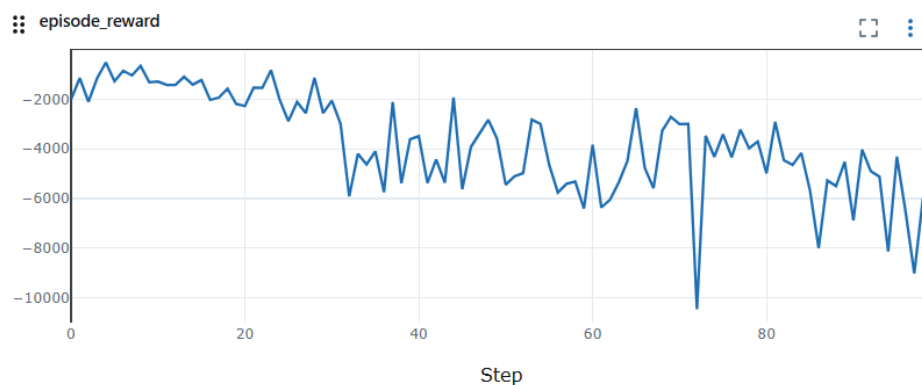
Foram conduzidas quatro execuções do agente DQN, variando batch size, fator de desconto e taxa de aprendizado e intensidade das penalidades de recompensa. O número de episódios treinados variou de 100 a 2000, com frequência de atualização da rede-alvo entre 10 e 100 episódios, e epsilon-greedy com ϵ inicial = 1.0 e ϵ mínimo $\in \{0.1, 0.01\}$ e decaimento entre 0.9999 e 0.995, garantindo alta exploração ao longo de quase todo o treinamento.

No primeiro treinamento, foi utilizado um batch size de 64, com $\gamma = 0.99$, taxa de aprendizado = 0.0001 e recompensas moderadas para ponto, game e set, resultando em

recompensas médias em torno de -460 e -495 e uma quantidade de ações. As tendências ascendentes na perda e nas ações ilegais, aliadas à tendência descendente da recompensa do agente, mostram que não houve um aprendizado significativo do agente em relação ao ambiente proposto.

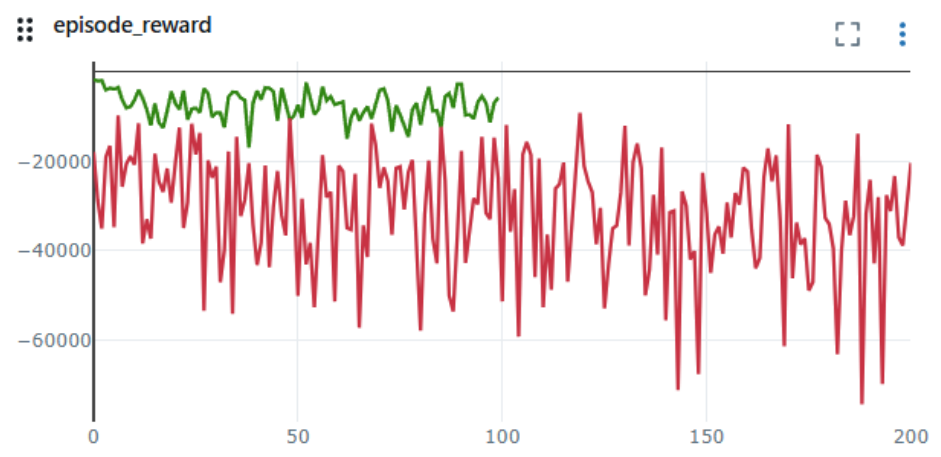


Em uma terceira execução, aumentou-se a severidade da penalização de ações ilegais para -5 e ajustaram-se recompensas de ponto para ± 0.1 e $l_r = 0.001$, o que levou a uma explosão da perda média e recompensas extremamente negativas (-3755), indicando instabilidade numérica e colapso dos Q-valores.



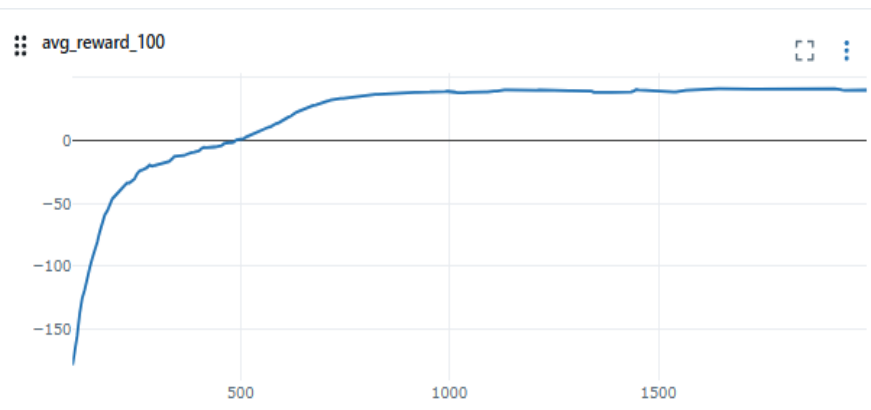
Nas duas últimas execuções, reduziu-se o batch size para 32, diminuiu-se γ para 0.95, endureceram-se drasticamente as recompensas de game e set (± 10 e ± 50) e o horizonte de treinamento foi ampliado para 200 episódios em um dos runs, mantendo $l_r = 0.001$ e forte penalização de ações ilegais (até -20). Mesmo assim, a recompensa média dos últimos 100 episódios permanece muito negativa (até -33704), o comprimento dos episódios permanece alto (até 1270 passos), a taxa de vitória acumulada continuou zero e o número de ações ilegais relatado no não chegou a cair para um patamar satisfatório,

confirmando que o DQN não aprendeu a evitar ações inválidas nem a explorar sequências de golpes vitoriosas.

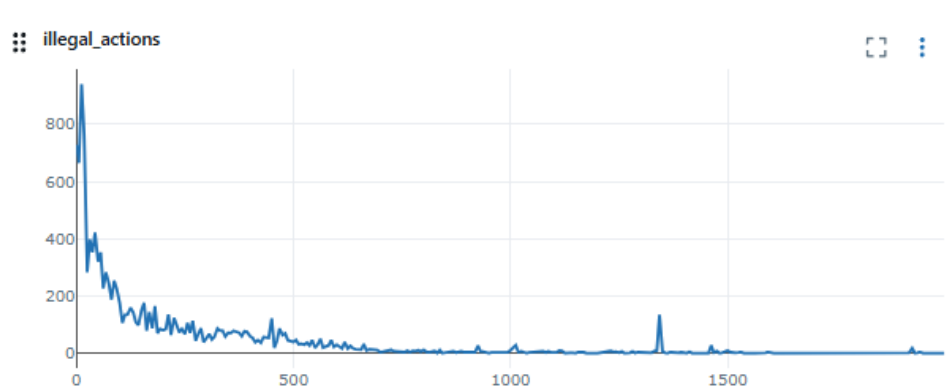


4.2 Reinforce

O algoritmo reinforce, com as configurações iniciais mencionadas na seção de metodologia, apresentou resultados aceitáveis, alcançando uma recompensa média positiva a partir dos 700 episódios e aprendendo a evitar ações ilegais, alcançando episódios com 0 ações inválidas executadas a partir do passo 850.



Recompensa Média dos últimos 100 episódios



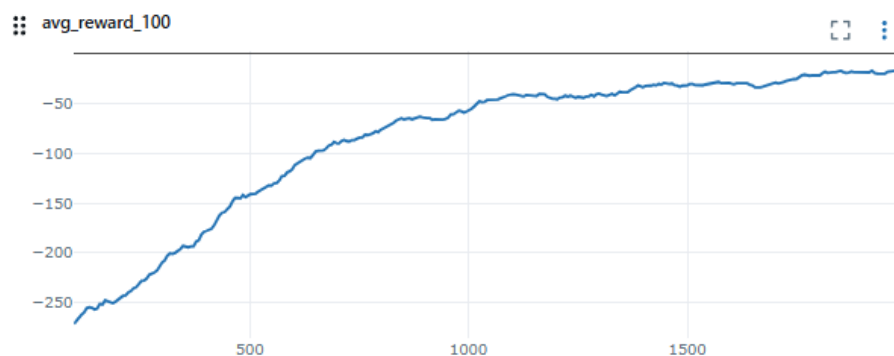
Quantidade de ações ilegais em cada episódio do treino

Adicionalmente, o agente treinado alcançou uma taxa de vitórias de 100% na fase de avaliação, demonstrando que foi aprendida uma estratégia muito eficiente para o ambiente proposto.

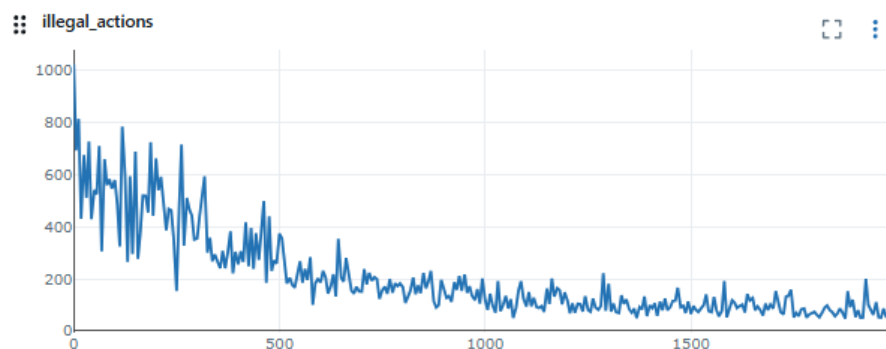
Foram experimentados dois learning rates adicionais com este algoritmo sendo estes:

- **Learning Rate = 0.0001**

Nesta configuração o modelo não foi capaz de convergir para uma solução ótima, apesar de mostrar um aprendizado lento ao longo dos episódios. A recompensa não alcançou a positividade devido à quantidade de ações ilegais que ainda estavam sendo executadas. Além disso, a taxa de vitórias do modelo final, na fase de avaliação, ficou em apenas 37%, evidenciando que não foram aprendidas estratégias vencedoras durante o treinamento.



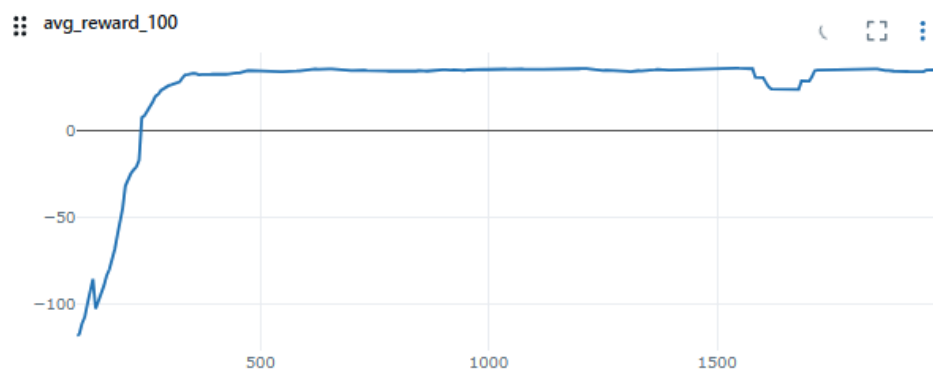
Recompensa Média dos últimos 100 episódios



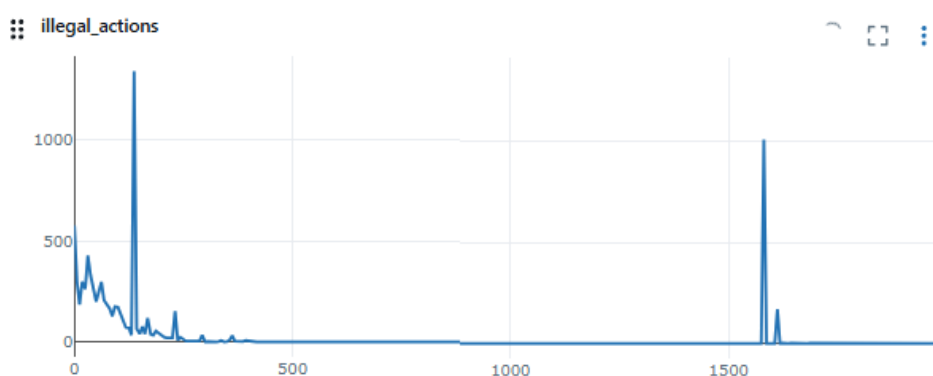
Quantidade de ações ilegais em cada episódio do treino

- **Learning Rate = 0.01**

Este hiperparâmetro mais agressivo trouxe uma convergência mais rápida para uma política ótima, apesar de o início do treinamento apresentar uma variação muito maior na perda, quando comparado aos outros experimentos. É possível perceber que tanto a recompensa quanto o número de ações inválidas melhoram rapidamente devido ao alto learning rate. A taxa de vitória alcançada ao final do treinamento foi de 100%, evidenciando que, além de executar ações válidas, o modelo aprendeu uma estratégia vitoriosa em partidas de tênis.



Recompensa Média dos últimos 100 episódios



Quantidade de ações ilegais em cada episódio do treino

5. Conclusão e Possíveis Melhorias

Por fim, o projeto demonstrou sucesso na implementação de um ambiente de simulação de tênis baseado em dados shot-by-shot do Match Charting Project, utilizando uma Rede de Markov para modelar transições de golpes e classes do Python para definir regras completas de pontuação. O agente REINFORCE superou o DQN, alcançando recompensa média positiva, zero ações inválidas e taxa de vitórias de 100% na avaliação, validando a abordagem de aprendizado por reforço para otimização tática em tênis.

A construção do ambiente simulou, de maneira simplificada, dinâmicas reais de partidas, com o REINFORCE adaptando-se bem à estocasticidade e ao grande espaço de ações (54 combinações de golpes e direções), enquanto o DQN enfrentou instabilidade devido a loops de ações ilegais e explosão de gradientes. Esses resultados destacam a superioridade de métodos baseados em política para cenários com alta variância e recompensas densas, oferecendo uma base para análises estratégicas em tênis profissional.

Melhorias futuras para o ambiente de simulação envolvem expandir a Rede de Markov para cadeias de 2ª ou 3ª ordem, incorporando histórico recente de golpes para capturar padrões táticos mais complexos e realistas observados em partidas reais. Além disso, o desenvolvimento de uma interface gráfica permitiria visualizar trajetórias de pontos, políticas aprendidas e comparações entre agentes, facilitando análises intuitivas para treinadores e jogadores.

Outras aprimorações incluem treinar agentes especializados contra estilos específicos de jogadores reais extraídos do dataset, simulando matchups personalizados, e aumentar a aleatoriedade com temperatura variável nas transições para maior variabilidade. Finalmente, mascarar ações ilegais diretamente na rede neural, adicionar regras contextuais avançadas (como proibir smashes em retornos curtos) e suportar múltiplos sets elevaria o realismo e a utilidade prática do sistema.

6. Referências

SACKMANN, J. Tennis Match Charting Project. GitHub, 2015. Disponível em: https://github.com/JeffSackmann/tennis_MatchChartingProject . Acesso em: 22 nov. 2025.

CHEN, H. et al. Reinforcement learning in tennis using shot-by-shot data. Journal of Exercise Science, [s.l.], 2023. Disponível em: <https://journal.esrgroups.org/jes/article/view/3125/2459> . Acesso em: 25 nov. 2025.

WILLIAMS, G. et al. Simple statistical modeling of tennis rallies. Stanford University, 2018. Disponível em: <https://web.stanford.edu/class/aa228/reports/2018/final84.pdf> . Acesso em: 25 nov. 2025.

HASSAN, H. et al. Design of tennis auxiliary teaching system based on reinforcement learning and multi-feature fusion. Heliyon, v. 10, n. 18, p. e12453859, 2024. Disponível em: <https://pmc.ncbi.nlm.nih.gov/articles/PMC12453859/> . Acesso em: 10 dez. 2025.

VAN HASSALT, H. et al. Deep reinforcement learning with double Q-learning. arXiv preprint arXiv:1509.06461, 2015. Disponível em: <https://arxiv.org/pdf/1509.06461.pdf> . Acesso em: 10 dez. 2025.